

be strictly adhered to, and the auto-generated code does not conform to your coding guidelines.

- Yours is an embedded systems project, which does not allow the use of recursion or allocation of dynamic memory, but the auto-generated code makes liberal use of them.
- Yours is a complex project, and you make use of auto-generators to specify the models. But the autogenerator does not support modelling a specific feature that is required for your project.

If you discover some of the limitations of the auto-generators late in the software development life-cycle, it can cause serious problems.

Now, after looking at the disadvantages and potential problems with auto-generating code, let's reflect on its advantages. Are there any? For example, I mentioned improved productivity as an advantage—is it really so? If you think about it, things like GUI builders help reduce the tedium involved in writing UI code. But we still take care of the harder part, which is design. For example, we need to separate the UI logic from business logic, and again maintain clean separation from the underlying database (or any data source). Design is hard work and takes time. In a large software development project, the time required for creating UI is negligible—it's the design during development, and re-factoring during maintenance that takes most of the time. So, the extent of 'improved productivity' may not be as significant as you would expect.

As you can see, here is a neutral view on auto-generating code, which boils down to a few fundamentally simple ideas:

- Auto-generating code helps improve productivity by simplifying the implementation of functionality, but it does not deal with the problem of meeting non-functional requirements.
- Auto-generating code, as a computer science problem, works well; but from a software

engineering perspective of not just developing, but also maintaining the software, it has its limitations. Though it solves many problems, it also introduces new problems to solve.

- Auto-generators help programmers like us to work on high-level abstractions; but any auto-generator tool will have specific limitations, and working around them will expose you to implementation details and may force you to work on the actual generated code.

So, if you're given a choice to either auto-generate the code or write it yourself from scratch, take a holistic look at auto-generating code, instead of blindly auto-generating the code.

Before I end this column, one final thought. Auto-generation is not just for source code, it could be used for generating test code as well. For example, with Model-Based Testing (MBT), you can specify the conditions to check in a diagrammatic form (i.e., a model), and the MBT tools would generate test cases for those conditions. It's an effective way to reduce unit testing effort and create effective test cases. Why? During testing, you don't systematically cover all the possibilities—and even for a few possibilities, the number of tests to be written to cover all those could be huge. But by auto-generating test code from models, this problem can be solved. However, this isn't used widely because it takes time to learn how to build models to generate test cases. Also, often the models generate test cases that will check for conditions that cannot occur in practice. So it takes time to find out which tests are legitimate and which tests are invalid. Even with these disadvantages, MBT is a good approach, and we should try to use it. 

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bengaluru. You can reach him at sgganesh@gmail.com.

Know the Leading Players in Every Sector of the Electronics Industry

www.electronicb2b.com

**ELECTRONICS
B2B INDUSTRY IS AT A**

CLICK OF A BUTTON

Log on to www.electronicb2b.com and be in touch with the Electronics B2B Fraternity 24x7